

GMCP-R：面向断续通信的历史连续性安全恢复协议

GMCP-R: A Memory-Continuity-Aware Secure Recovery Protocol for Intermittent Communications

Abstract (摘要)

连接恢复并不等同于历史状态恢复。针对断续通信中恢复后历史连续性难以验证的问题，本文提出 GMCP-R (Memory-Continuity-Aware Communication Protocol with Recovery)。GMCP-R 解决的是断连或攻击检测后“如何确认恢复后的通信历史仍然连续可信”的问题；其恢复目标不是单纯重新建立 TCP 连接，而是恢复由 last_seq、last_mem 与 Checkpoint 共同刻画的历史记忆状态。具体而言，last_seq 表示可接受的恢复点，last_mem 表示截至该恢复点处理后的链式历史状态，Checkpoint 提供最近可信快照以限制恢复重放范围。该协议结合 memory chain、MemoryTicket 与 Checkpoint：memory chain 将消息内容、序列号、会话上下文和前序记忆状态绑定；MemoryTicket 以服务器认证票据保存 last_seq、last_mem 与 checkpoint 信息；Checkpoint 用于限制恢复时的历史重放范围。基于 Python TCP 原型的实验表明，在测试攻击场景下，GMCP-R 达到 100% 检测率、0% 误接受率和 0% 误拒绝率，五类无效 MemoryTicket 均被拒绝；本地纯计算吞吐量均值为 65,931 msg/s，1 至 20 个并发客户端均保持 100% 成功率，吞吐量峰值为 17,927 msg/s。弱网结果仅来自 preliminary code-level simulation，用于补充观察恢复行为，不作为核心结论。结果说明，GMCP-R 可在可接受开销下为断续通信提供历史连续性验证与安全恢复支持。

Keywords (关键词)：安全通信；历史连续性；状态恢复；断续通信；MemoryTicket；Checkpoint；攻击检测；通信协议

1. Introduction (引言)

1.1 研究背景

现代网络系统越来越多地运行在不稳定链路、移动终端、工业现场网络、远程设备和 IoT 场景中。连接中断、短时丢包、设备切换和攻击检测后的主动断开，都会迫使通信双方执行恢复过程。TLS 1.3、QUIC 等协议已经提供成熟的连接建立和会话恢复机制，但这些机制主要解决“如何重新建立可通信信道”的问题，而不直接回答“恢复后的状态是否仍然代表一段连续、完整且未被篡改的通信历史”。

对于普通即时通信或一般请求响应系统而言，重新连接后继续发送消息可能已经足够；但在金融交易流水、医疗记录同步、远程控制、审计日志、工业数据采集等场景中，历史状态本身就是安全对象。攻击者

即使无法长期控制端点，也可能通过丢弃、篡改、重放或伪造历史记忆字段，使通信双方在恢复后进入不一致状态。

1.2 问题定义：恢复连接不等于恢复历史

本文关注断续通信中的历史连续性恢复问题：当通信在第 i 条消息附近发生异常中断或攻击检测后，恢复机制不仅应恢复会话，还应验证恢复点之前的消息历史与双方维护的记忆状态一致。换言之，恢复后的 $last_seq$ 与 $last_mem$ 必须能解释为同一条历史链上的状态，而不是由攻击者诱导出的旧状态、跨会话状态或伪造状态。

在本文中，恢复点由 $last_seq$ 表示，恢复状态由 $last_mem$ 表示。 $last_seq$ 表示通信双方已经接收并接受的最新消息序号， $last_mem$ 表示截至该序号处理后的历史记忆状态；Checkpoint 则记录最近可信的 $\{ckpt_seq, ckpt_mem\}$ 快照，用于缩短恢复时需要验证或重放的历史窗口。一次安全恢复并不是简单重连，而是要求恢复后的 $last_seq$ 与 $last_mem$ 能够对应同一条合法历史链。若攻击者试图使用旧票据、跨会话票据或伪造记忆状态恢复通信，则服务端应拒绝该恢复请求。

表 1. 现有方案与 GMCP-R 的能力对比

方法	连接恢复	历史验证	状态恢复	主要局限
TLS/QUIC 会话恢复	支持	不直接支持	恢复加密上下文	不验证断连期间历史连续性
Seq+MAC	不直接支持	部分支持	不支持	缺少记忆链，无法检测 $prev_mem$ 伪造
Hash Chain	不直接支持	支持	需要历史重放	长会话恢复开销随历史线性增长
Ticket Only	支持	不支持	恢复会话票据	缺少消息内容与历史状态绑定
GMCP-R	支持	支持	支持	以适度计算开销实现可验证恢复

1.3 Research Gap（研究空白）

现有轻量认证与恢复方案各自覆盖了断续通信安全问题的一部分，但缺少面向历史连续性恢复的统一机制。Seq+MAC 主要解决单包完整性与序列保护，难以表达跨消息历史状态；Ticket Only 能支持连接或会话恢复，却不验证恢复点之前的消息历史；Hash Chain 可以验证历史篡改，但单独使用时缺少安全恢复票据和低成本恢复路径。因此，仍需要一种机制同时支持历史连续性验证、安全恢复票据、防回滚、防重放以及低成本恢复。

1.4 本文贡献

1. 定义断续通信中的历史连续性恢复问题，区分连接恢复、消息完整性验证与历史状态恢复，并明确恢复点 $last_seq$ 与恢复状态 $last_mem$ 的语义。
2. 提出基于 memory chain 的 GMCP-R 协议，将会话、时代号、序列号、载荷哈希和发送方标识绑定到逐消息记忆状态。

3. 设计 MemoryTicket 与 Checkpoint 组合机制，使恢复请求同时具备签名完整性、时效性、一次性 nonce、防回滚和会话绑定能力。
4. 实现 Python TCP 原型，并在真实 baseline 对比、无效票据拒绝、本地性能、并发客户端和代码级弱网仿真中评估协议表现。

2. Related Work (相关工作)

2.1 消息认证与序列保护

消息认证码广泛用于保护消息完整性与来源真实性。HMAC 将共享密钥与消息内容绑定，接收方可验证消息是否在传输中被篡改 [REF-HMAC]。序列号机制则用于检测重放、乱序或旧消息注入。Seq+MAC 类方案将二者组合后能够保护单条消息及其顺序，但如果协议没有维护跨消息的历史记忆状态，攻击者仍可能围绕 prev_mem 或恢复状态构造攻击。

2.2 TLS/QUIC 会话恢复

TLS 1.3 和 QUIC 支持通过 PSK、0-RTT 等机制快速恢复通信上下文 [REF-TLS13], [REF-QUIC]。这些机制强调低时延重连和密钥上下文恢复，但通常不将应用层历史状态作为恢复验证对象。因此，它们不能直接替代本文讨论的历史连续性恢复机制，而更适合作为 GMCP-R 未来集成的底层安全信道。

2.3 哈希链与可篡改检测日志

哈希链在一次性口令、认证序列和安全日志中被广泛使用 [REF-HASHCHAIN]。在通信历史验证中，哈希链可以将每条消息纳入可篡改检测的链式承诺；任何历史消息被修改都会影响后续状态。然而，纯 Hash Chain 在恢复时通常需要从初始状态或较早状态重放历史，长会话场景下恢复开销较高。GMCP-R 继承哈希链的历史绑定能力，同时通过 Checkpoint 与 MemoryTicket 降低恢复成本。

2.4 Checkpoint 与断点恢复

Checkpoint 技术在分布式系统和容错计算中常用于保存可恢复状态。将其引入通信历史验证后，协议无需每次从会话初始点重建记忆状态，而可以从最近的可信快照恢复。GMCP-R 的 Checkpoint 不仅保存序列号和记忆值，还与 MemoryTicket 中的恢复字段共同接受签名和一致性检查。

2.5 安全协议与新型网络系统

Noise、WireGuard、Signal Double Ratchet、命名数据网络和形式化协议分析等工作从不同角度处理密钥更新、链路认证、状态演进、内容命名或协议验证。这些研究说明“状态如何随消息演进”是安全协议设计中的重要问题。GMCP-R 的目标不是替代这些协议，而是在断续通信恢复过程中提供一层可审计的历史连续性验证机制。

2.6 State Continuity and Rollback Protection

状态连续性与防回滚问题在可信计算、安全存储和 TEE 场景中已有较多研究。传统方案通常依赖 TPM 单调计数器、可信时间源、NVRAM、TEE 或受保护持久化存储，以确保安全状态不会被攻击者回退到旧版本。这类机制适合强可信硬件环境，但在断续通信、IoT 边缘设备和轻量级应用层协议中，不一定具备低成本、跨平台和易部署的条件。

GMCP-R 不替代硬件级防回滚机制，而是在通信协议层提供恢复过程中的状态连续性检查。具体而言，MemoryTicket 将 last_seq、last_mem、checkpoint、nonce 和 session 信息绑定到服务器认证标签中，使服务端在本文威胁模型下检测恢复阶段的票据重放、历史回滚和跨会话恢复攻击。该设计可与可信硬件机制互补，也可在缺少专用硬件支持的轻量场景中提供应用层恢复验证。

3. System Model and Security Goals（系统模型与安全目标）

3.1 系统实体与通信状态

系统包含客户端 C、服务器 S 以及网络攻击者 A。客户端与服务器通过 TCP 连接传输 DATA 报文，并共同维护会话标识 session_id、发送方标识 sender_id、时代号 epoch、单调递增序列号 seq、载荷哈希 payload_hash、前一记忆状态 prev_mem、当前记忆状态 last_mem 以及恢复所需的 Checkpoint 与 MemoryTicket。

表 2. GMCP-R 的主要安全目标

安全目标	含义
消息完整性	任何对 payload 或关键元数据的篡改都应触发认证失败
历史连续性	消息历史应形成可验证的链式状态，prev_mem 伪造应被检测
重放抵抗	旧消息或旧票据不能被再次接受当前有效状态
回滚抵抗	恢复过程不能使双方退回到低于当前要求的 last_seq 或旧 last_mem
票据真实性	MemoryTicket 必须由服务器签发，字段被改动后应无法通过验证
恢复后一致性	恢复后的 last_seq 与 last_mem 应与已接受历史相匹配

3.2 攻击模型

本文采用 Dolev-Yao 风格的网络攻击者模型，后续可结合形式化工具进一步建模 [REF-PROVERIF], [REF-TAMARIN]。攻击者可以拦截、删除、修改、重放或注入网络消息，也可以尝试构造过期、重放、篡改、回滚或跨会话的 MemoryTicket。攻击者不能破坏 SHA-256、HMAC-SHA256 等密码学原语，不能获得通信双方共享密钥，也不能控制客户端或服务器端点。本文所有安全结论均限定在该威胁模型和已测试攻击场景之内。

4. Proposed GMCP-R Protocol（协议设计）

4.1 协议概览

GMCP-R 在正常通信阶段对每条消息进行载荷哈希、记忆链更新和 HMAC 认证；在运行过程中周期性创建 Checkpoint；当连接中断或攻击被检测后，客户端携带 MemoryTicket 发起恢复请求，服务器验证票据、上下文和状态单调性后返回可恢复状态。协议设计的核心思想是：恢复票据不是单纯的会话凭证，而是经过服务器签名的历史状态承诺。

从整体流程看，GMCP-R 将正常 DATA 通信、周期性 Checkpoint、MemoryTicket 签发与异常后的恢复请求放在同一状态框架内。DATA 阶段持续推进 last_seq 与 last_mem，恢复阶段则以 MemoryTicket 和 Checkpoint 验证恢复点及恢复状态是否仍属于同一条合法历史链。

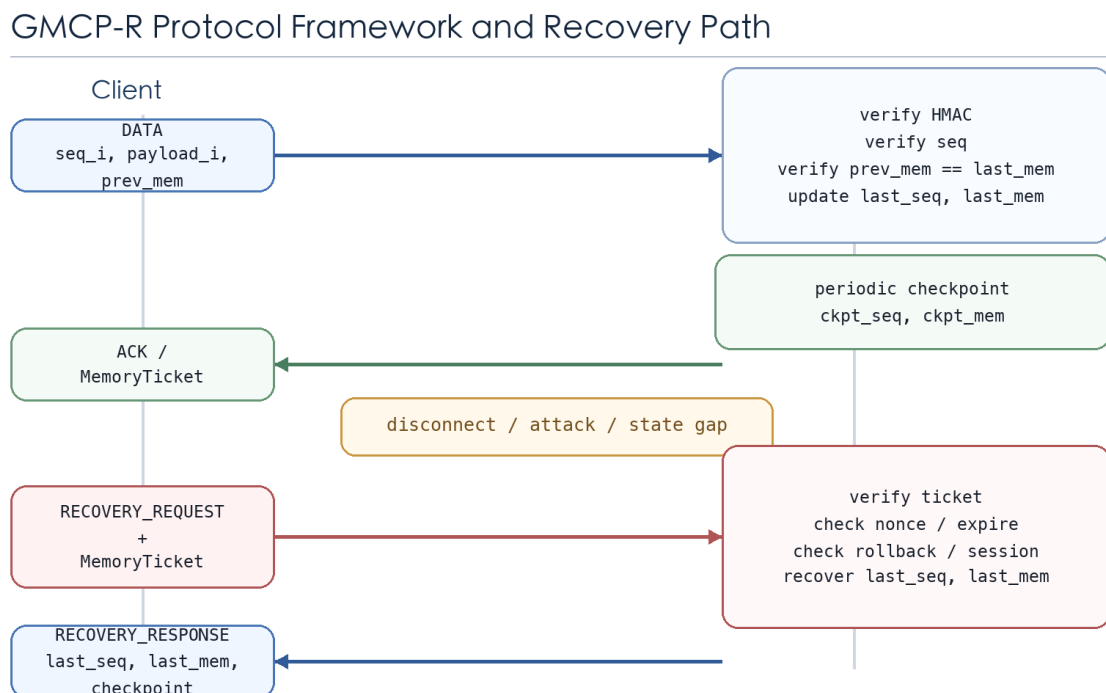


图 1. GMCP-R 协议整体框架与恢复路径。

4.2 Memory Chain

记 M_i 表示第 i 条消息处理后的记忆状态， h_i 表示 payload_i 的哈希。GMCP-R 的记忆链更新可形式化表示为：

$$M_i = H(M_{i-1} || \text{sid} || e || \text{seq}_i || h_i || \text{sender})$$

$$h_i = H(\text{payload}_i)$$

其中， sid 表示会话标识， e 表示时代号， seq_i 表示第 i 条消息的单调序列号， sender 表示发送方标识， H 表示密码学哈希函数。与仅使用 payload 的简单哈希链相比，该定义将消息放入具体会话与序列上下文中，可降低跨会话混淆、乱序注入和历史回滚的空间。在哈希函数满足抗碰撞性和原像抗性的假设下，攻击者难以构造不同历史却得到相同 M_i ，也难以从当前记忆状态反推出可替换的历史前缀。因此，任何历史字段被修改都会影响后续记忆状态，并在 prev_mem 或 last_mem 一致性检查中暴露。

图 2 展示了 memory chain 的状态演进关系。每条消息均携带发送前的 prev_mem ，接收端只有在 prev_mem 与本地 last_mem 一致时才接受该消息，并在认证通过后计算新的 last_mem 。

GMCP-R Memory Chain State Evolution

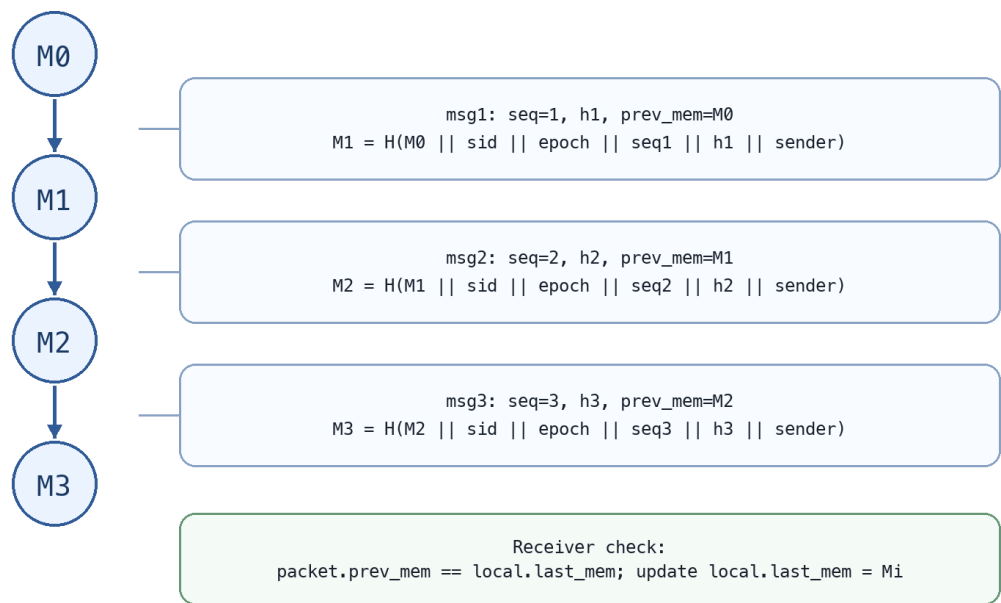


图 2. GMCP-R 的 memory chain 状态演进过程。

4.3 DATA 报文格式

表 3. GMCP-R DATA 报文字段

字段类别	字段	作用
会话上下文	session_id, sender_id, epoch	将消息绑定到具体会话、发送方和状态阶段，防止跨会话或跨阶段混淆。
顺序状态	seq	保证消息单调递增，检测重放、旧包和序列缺口。
内容绑定	payload, payload_hash	将应用层内容绑定到认证计算与记忆链更新中。
链式状态	prev_mem	表示发送前的前序记忆状态，用于与接收端本地 last_mem 比较，验证历史连续性。
链式结果	next_mem / curr_mem（可选）	表示发送方计算出的消息后记忆状态；接收端可重新计算并比对，适用于调试、审计或跨节点同步。
时间辅助	timestamp	辅助检测过期、异常延迟或审计排序。
认证字段	auth_tag	对会话上下文、顺序状态、内容绑定和链式状态等关键字段进行 HMAC

字段类别	字段	作用
		认证。

其中，prev_mem 是 DATA 报文中必需的链式状态输入，接收端必须验证 $P.\text{prev_mem} == \text{local.last_mem}$ ；随后接收端根据 payload_hash、seq、session_id、epoch 和 sender_id 重新计算 M_i 。next_mem / curr_mem 属于可选扩展字段，在当前原型中可由接收端本地计算得到，因此不作为必需传输字段；若显式携带，接收端应重新计算 M_i 并在 $P.\text{next_mem} \neq M_i$ 时拒绝该报文。

$\text{auth_tag} = \text{HMAC}(K, \text{packet_without_auth})$

Algorithm 1. GMCP-R DATA Packet Verification

Input: packet P, expected_session_id, last_seq, last_mem, key K
Output: accept/reject decision and updated memory state
1: if required fields are missing then reject
2: if P.type != DATA then reject
3: if P.session_id != expected_session_id then reject
4: $h' = H(P.\text{payload})$
5: if $h' \neq P.\text{payload_hash}$ then reject
6: $\text{tag}' = \text{HMAC}(K, P \text{ without auth_tag})$
7: if $\text{tag}' \neq P.\text{auth_tag}$ then reject
8: if $P.\text{seq} \leq \text{last_seq}$ then reject as replay or stale packet
9: if $P.\text{seq} > \text{last_seq} + 1$ then reject as sequence gap
10: if $P.\text{prev_mem} \neq \text{last_mem}$ then reject as history discontinuity
11: $M_i = H(\text{last_mem} \parallel P.\text{session_id} \parallel P.\text{epoch} \parallel P.\text{seq} \parallel P.\text{payload_hash} \parallel P.\text{sender_id})$
12: if P.next_mem is present and $P.\text{next_mem} \neq M_i$ then reject as memory-result mismatch
13: update $\text{last_seq} = P.\text{seq}$ and $\text{last_mem} = M_i$
14: accept P

4.4 Checkpoint 机制

Checkpoint 是周期性状态快照，记录 session_id、epoch、seq、memory、timestamp 与签名。默认配置下每 100 条消息创建一个 Checkpoint。恢复时，协议可从最近 Checkpoint 继续重放少量后续消息，而无需从会话起点重建全部历史，从而将恢复开销由 $O(n)$ 限制为 $O(k)$ ，其中 k 为 Checkpoint 间隔。

4.5 MemoryTicket 机制

表 4. MemoryTicket 核心字段与安全作用

字段	作用
session_id / client_id / epoch	绑定恢复请求的会话、客户端和时代上下文
last_seq / last_mem	记录票据签发时的最新序列号与记忆状态
ckpt_seq / ckpt_mem	记录最近可信快照，支持有界恢复
expire_time	限制票据有效期，防止期用
ticket_nonce	一次性随机数，用于检测票据重放
key_version	支持后续密钥轮换或版本管理
server_auth_tag	服务器 HMAC 签名，保护票据字段完整性

MemoryTicket 的认证标签覆盖恢复相关字段，可形式化表示为：

$\text{tag_T} = \text{HMAC}_{\{K_s\}}(\text{sid} \parallel \text{cid} \parallel \text{e} \parallel \text{last_seq} \parallel \text{last_mem} \parallel \text{ckpt_seq} \parallel \text{ckpt_mem} \parallel \text{exp} \parallel \text{nonce} \parallel \text{kv})$

其中，K_s 为服务器侧票据认证密钥，cid 表示客户端标识，exp 表示过期时间，nonce 表示一次性票据随机数，kv 表示密钥版本。由于 tag_T 覆盖会话、客户端、时代号、恢复序列、记忆状态、checkpoint、过期时间、nonce 和密钥版本，攻击者修改任一字段都会导致服务器重新计算的认证标签不匹配，从而使恢复请求被拒绝。

4.6 恢复流程

1. 客户端检测到断连、序列号间隙或攻击响应后，准备最近一次收到的 MemoryTicket。
2. 客户端发送 RECOVERY_REQUEST，并附带 ticket、session_id、client_id、epoch 等上下文。
3. 服务器验证票据字段、HMAC 签名、过期时间、nonce 是否已使用、session_id/client_id/epoch 是否匹配，以及 last_seq 是否满足单调性约束。
4. 验证通过后，服务器返回恢复状态 (last_seq、last_mem、checkpoint_seq、checkpoint_mem)；验证失败时返回明确拒绝原因，并拒绝更新状态。
5. 客户端从 last_seq + 1 继续通信，并用恢复后的 last_mem 作为后续 memory chain 的起点。

因此，GMCP-R 的恢复对象不是应用层 payload 全文，也不是单纯网络连接句柄，而是能够作为后续安全通信起点的历史记忆状态：last_seq 给出恢复点，last_mem 给出该恢复点对应的链式状态，Checkpoint 给出可验证的近端快照。三者一致时，通信双方才能继续推进新的 DATA 报文。

Algorithm 2. MemoryTicket-Based Recovery

Input: recovery request R, MemoryTicket T, server state S
Output: recovery response or explicit rejection reason
1: check required ticket fields are present; otherwise reject with "missing ticket field"
2: verify T.server_auth_tag using server key; otherwise reject with "invalid ticket auth tag"
3: if T.expire_time < current_time then reject with "ticket expired"
4: if T.ticket_nonce has been consumed then reject with "ticket replay detected"
5: if T.session_id, T.client_id, or T.epoch mismatches R then reject with "session_id/client_id/epoch mismatch"
6: if T.last_seq < S.min last_seq then reject with "rollback detected"
7: if T.checkpoint_seq > T.last_seq then reject with "checkpoint_seq exceeds last_seq"
8: consume T.ticket_nonce
9: return recovery state (last_seq, last_mem, checkpoint_seq, checkpoint_mem)

5. Security Analysis（安全分析）

5.1 消息完整性

Claim. Message Integrity. 在 HMAC-SHA256 满足不可伪造性的假设下，攻击者难以修改 DATA 报文中的 payload、seq、prev_mem 或会话上下文字段而仍使接收端接受该报文。

Proof Sketch. 每个 DATA 报文的 auth_tag 覆盖除认证标签自身外的关键字段，包括 payload_hash、seq、prev_mem、session_id、epoch 与 sender_id。攻击者若修改任一字段，则服务器重新计算得到的 HMAC 输

入发生变化。若修改后仍能通过验证，则意味着攻击者在未知共享密钥 K 的情况下伪造了有效 HMAC 标签，与 HMAC 不可伪造性假设矛盾。

5.2 历史连续性

Property 1. History Continuity. 在哈希函数 H 满足抗碰撞性，且 HMAC 不可伪造的假设下，攻击者难以构造一条与已接受历史不同的消息序列，却使接收端得到相同且可接受的 `last_mem`。

Proof Sketch. 设两条历史序列在某一位置 j 首次不同，但最终均被接收端接受并得到相同记忆状态 M_i 。由于 M_i 递归依赖 $M_{\{i-1\}}$ 、`session_id`、`epoch`、`seq_i`、`payload_hash_i` 和 `sender_id`，位置 j 的差异会改变后续所有记忆状态输入。若最终状态仍相同，则要么存在一次哈希碰撞，要么攻击者伪造了覆盖差异字段的 HMAC 标签。根据 H 的抗碰撞性和 HMAC 的不可伪造性，该事件概率可忽略。因此，不同历史难以产生相同可接受记忆状态。

5.3 票据真实性与恢复后一致性

Property 2. Ticket Unforgeability and Recovery Consistency. 在 HMAC 安全假设下，攻击者不能修改 MemoryTicket 中的 `last_seq`、`last_mem`、`checkpoint`、`expire_time`、`nonce` 或上下文字段而仍通过服务端验证；通过验证的票据必须描述同一会话、同一客户端、同一 epoch 下的一致恢复状态。

Proof Sketch. MemoryTicket 的 `server_auth_tag` 覆盖 `sid`、`cid`、`epoch`、`last_seq`、`last_mem`、`ckpt_seq`、`ckpt_mem`、`exp`、`nonce` 和 `key_version`。攻击者修改任一字段都会改变 HMAC 输入；若修改后仍通过验证，则等价于成功伪造 HMAC 标签。恢复算法还显式检查 `session_id`、`client_id` 与 `epoch` 是否匹配请求上下文，并检查 `checkpoint_seq` $\leq$ `last_seq`。因此，服务端只接受与当前恢复请求一致、且结构上合法的恢复状态。

5.4 重放与回滚抵抗

Property 3. Rollback Resistance. 若服务端维护最小可接受恢复序号 `min_last_seq`，并记录已消费 `ticket_nonce`，则任何 `last_seq` $<$ `min_last_seq` 的旧票据或重复使用票据均会被拒绝。

Proof Sketch. 服务端在恢复算法中显式检查 `T.last_seq` $<$ `S.min_last_seq`，并在该条件成立时返回 `rollback detected`；同时，已消费的 `ticket_nonce` 会被记录并用于拒绝重复恢复请求。因此，攻击者无法利用旧票据将服务端恢复到低于 `min_last_seq` 的历史位置，也无法通过重复提交同一有效票据绕过状态单调性检查。报文级重放还会被 `seq` $\leq$ `last_seq` 检测，避免恢复后接受旧 DATA 报文。

需要强调的是，本文目前提供的是工程实现与实验场景下的安全分析，而非完整形式化证明。更严格的证明可在后续工作中借助 ProVerif、Tamarin 或类似工具完成 [REF-PROVERIF], [REF-TAMARIN]。

6. Implementation and Experimental Setup（实现与实验设置）

6.1 原型实现

项目使用 Python 3.11 实现 GMCP-R 原型，核心模块包括 memory.py、packet.py、ticket.py、checkpoint_manager.py 与 protocol.py。密码学操作采用 SHA-256 与 HMAC-SHA256。实验包含真实 TCP baseline 对比、MemoryTicket 无效票据拒绝、本地纯计算性能、并发客户端测试和代码级弱网仿真。

表 5. 实验环境与参数概览

类别	配置
客户端环境	Python 3.11, macOS / Darwin 24.6.0（部分实验元数据记录）
服务器环境	Linux CentOS-7.6.1810-x64, 4 核 CPU, 4GB 内存, VPC 网络
传输协议	TCP/IP；GMCP-R 默认端口 9000, baseline 默认端口 9001
基线协议	Hash Chain、Seq+MAC、Ticket Only
消息数量	100、500、1000；并发实验为每客户端 200 条
载荷大小	64、128、256、512、1024 bytes（按实验类型不同取子集）
攻击类型	drop、modify、replay、prev_mem；票据实验另含 expired、replayed、tampered、rollback、wrong_session
主要指标	吞吐量、RTT、攻击检测率、误接受率、误拒绝率、恢复成功率、并发成功率

数据来源：docs/experiment_methodology.md、results/**/*.csv 与脚本元数据。

6.2 Baseline 与测试矩阵

真实 baseline 对比实验覆盖 4 种协议、5 类攻击类型、3 种消息数量、2 种载荷大小与 3 次重复，共 360 条记录。MemoryTicket 实验覆盖 5 类无效票据、2 种消息数量与 3 次重复，共 30 条记录。弱网仿真覆盖 3 种协议、5 档包率与 5 档延迟，结果作为初步恢复能力验证，而非真实网络损伤结论。

7. Results（实验结果）

7.1 Baseline Comparison

表 6 汇总了真实 baseline 对比实验的主要结果。GMCP-R 与 Hash Chain 在测试攻击场景中均达到 100% 攻击检测率和 0% 误接受率；Seq+MAC 因缺少历史记忆链，对 prev_mem 类攻击存在缺口；Ticket Only 仅保护会话票据，无法完整验证消息内容与历史连续性。

表 6. Baseline 协议对比结果

协议	正常吞吐量 (msg/s)	正常 RTT (ms)	攻击检测率	误接受率	误拒绝率
GMCP-R	9,621	0.103	100%	0%	0%
Hash Chain	11,786	0.096	100%	0%	0%
Seq+MAC	11,727	0.091	75%	25%	0%
Ticket Only	13,376	0.087	50%	50%	0%

数据来源：results/real_baseline_comparison/summary_real_baseline_comparison_v2.csv。

本节采用与表 6 一致的 v2 汇总口径报告描述性统计。可以看到，GMCP-R 与 Hash Chain 在测试攻击集合中的检测率均为 100%，而 Seq+MAC 和 Ticket Only 分别存在 25% 与 50% 的误接受率。考虑到当前真实

baseline 每个配置的重复次数仍较少，本稿暂不报告正式统计推断，后续扩展重复次数后再补充显著性检验。

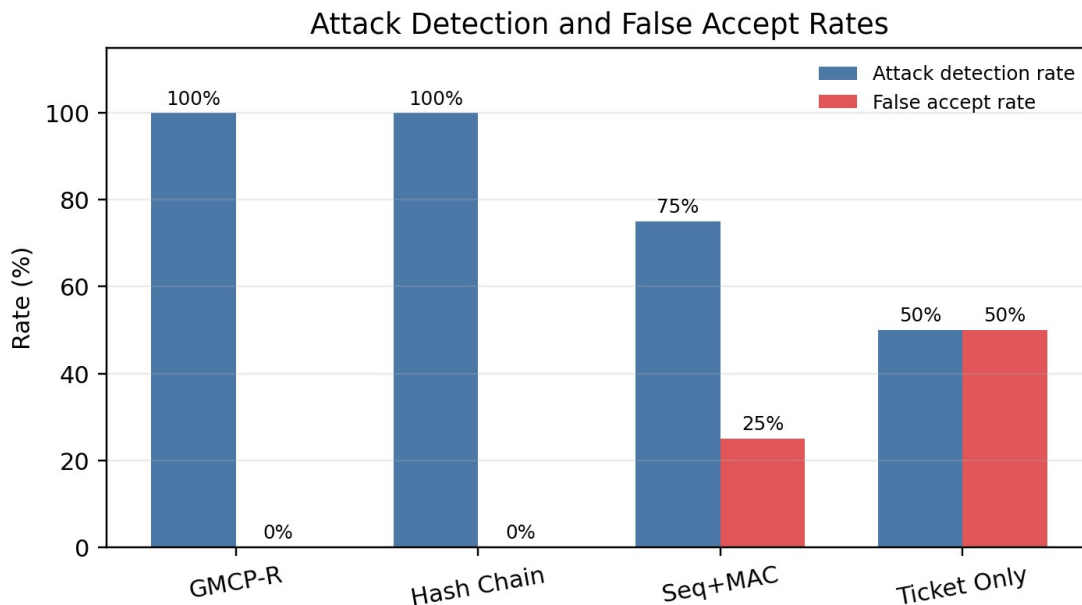


图 3. 各协议攻击检测率与误接受率对比。

数据来源：results/real_baseline_comparison/summary_real_baseline_comparison_v2.csv。

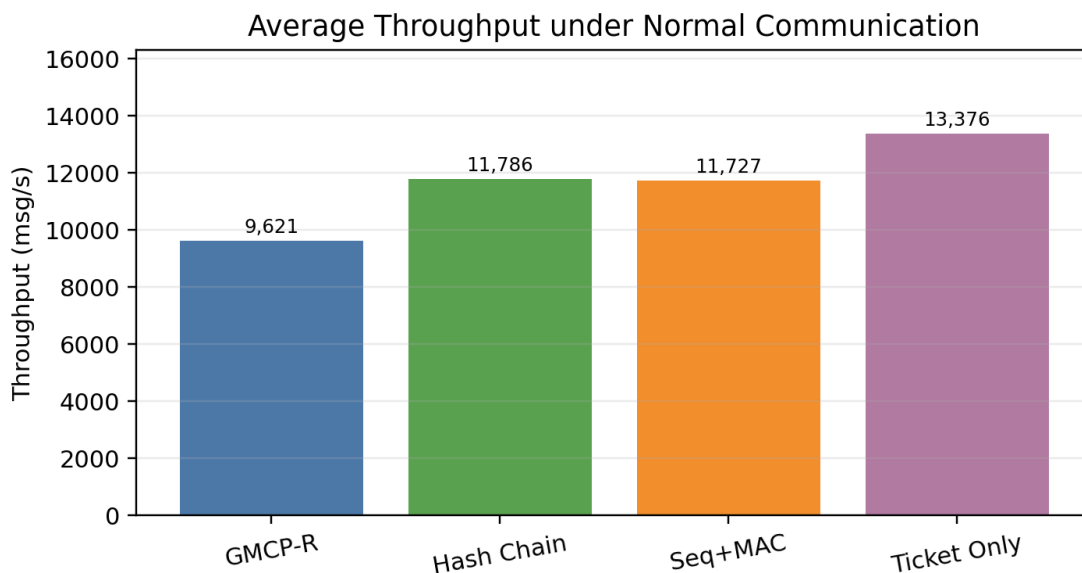


图 4. 正常通信条件下各协议平均吞吐量对比。

数据来源：results/real_baseline_comparison/summary_real_baseline_comparison_v2.csv。

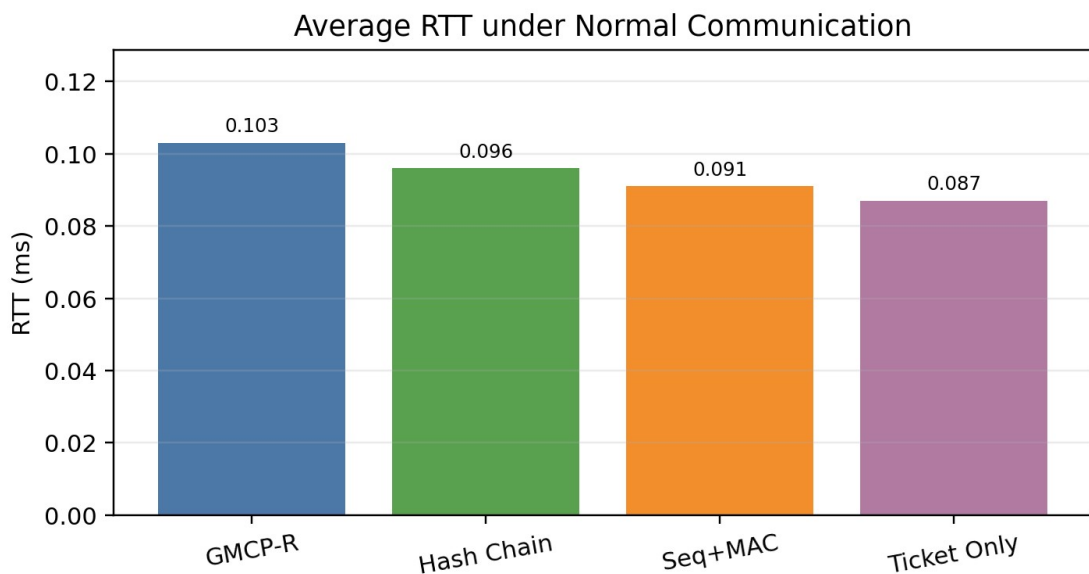


图 5. 正常通信条件下各协议平均 RTT 对比。

数据来源：results/real_baseline_comparison/summary_real_baseline_comparison_v2.csv。

上述差异主要来自各协议绑定历史状态的能力不同。Seq+MAC 能验证单包 HMAC 和序列单调性，因此可检测 payload 篡改、旧序列重放等攻击，但它不维护跨消息 memory chain；当攻击围绕 prev_mem 或历史连续性构造时，接收端缺少可比较的链式状态，因而出现误接受。Ticket Only 更偏向恢复凭证验证，能够表达“某个票据是否有效”，但不将每条 DATA 报文的载荷、序列和历史记忆状态绑定到恢复检查中，因此误接受率更高。GMCP-R 与 Hash Chain 在测试攻击集中的检测率相同，二者都依赖链式历史状态；GMCP-R 的区别不在于检测率高于 Hash Chain，而在于额外提供 MemoryTicket 与 Checkpoint 支持的安全恢复机制。

7.2 Invalid MemoryTicket Rejection

为验证恢复票据机制的安全性，实验构造了过期、重放、篡改、回滚和跨会话五类无效 MemoryTicket。结果表明，所有无效票据均被服务器拒绝，并返回明确拒绝原因。

表 7. MemoryTicket 无效票据拒绝结果

票据类型	恢复成功率	服务器拒绝原因	验证机制
expired_ticket	0% (全部拒绝)	ticket expired	过期时间校验
replayed_ticket	0% (全部拒绝)	ticket replay detected; rollback detected in repeated-use cases	nonce 唯一性与状态单调性校验
tampered_ticket	0% (全部拒绝)	invalid ticket auth tag	HMAC 签名校验
rollback_ticket	0% (全部拒绝)	checkpoint_seq exceeds last_seq	结构一致性与防回滚校验
wrong_session_ticket	0% (全部拒绝)	session_id mismatch	会话标识绑定校验

数据来源：results/real_ticket_recovery/real_ticket_recovery_results.csv。

五类无效票据分别对应恢复机制中的关键安全检查：expired_ticket 检查票据时效性，replayed_ticket 检查 nonce 一次性与状态单调性，tampered_ticket 检查服务器认证标签，rollback_ticket 检查 checkpoint 与 last_seq 的结构一致性，wrong_session_ticket 检查会话绑定。实验结果说明，在这些测试场景下，MemoryTicket 不只是恢复凭证，也承担恢复状态完整性、重放检测和防回滚检查的入口。

7.3 Performance Benchmark

本地纯计算性能基准测试排除了网络往返影响，主要衡量协议构造、认证和验证逻辑的开销。GMCP-R 的平均吞吐量约为 65,931 msg/s，平均端到端处理延迟约为 14.96 μ s。与更轻量的 Ticket Only、Hash Chain 和 Seq+MAC 相比，GMCP-R 的开销主要来自 payload_hash、memory chain 更新、字段覆盖更完整的 HMAC 以及 Checkpoint 管理。

表 8. 性能基准测试结果（本地回环，64B-1024B 均值）

协议	平均吞吐量 (msg/s)	平均端到端延迟 (μ s)
GMCP-R	65,931	14.96
Hash Chain	260,437	3.45
Seq+MAC	213,051	4.26
Ticket Only	364,572	2.29

数据来源：results/performance/performance_benchmark_results.csv。

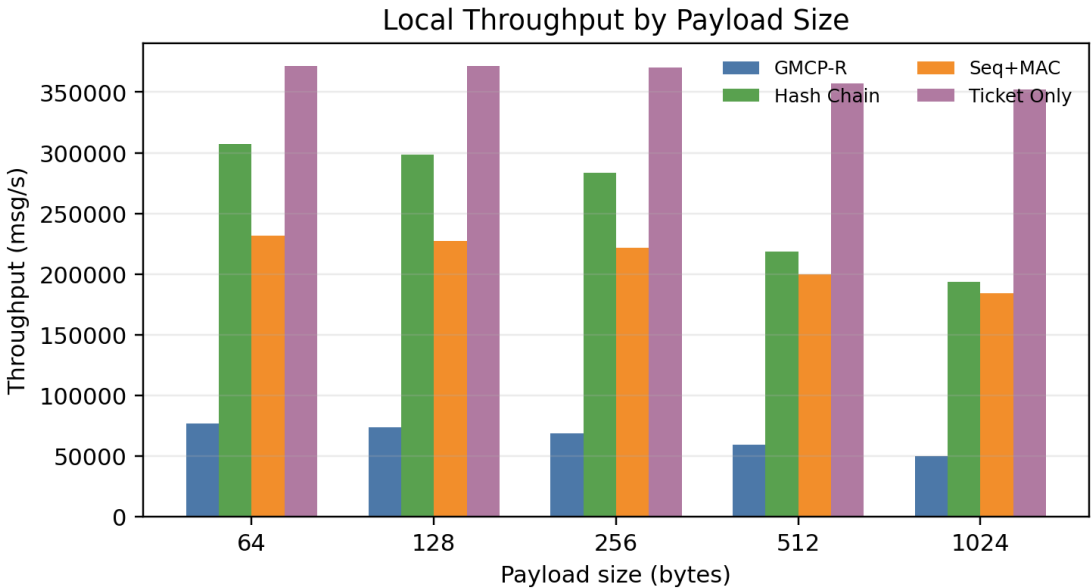


图 6. 本地纯计算吞吐量随载荷大小变化。

数据来源：results/performance/performance_benchmark_results.csv。

性能结果体现了 GMCP-R 的主要权衡：协议用额外计算开销换取历史连续性验证和恢复状态绑定。相较于 Ticket Only，GMCP-R 每条消息需要计算 payload_hash、更新 memory state，并对更多上下文字段进行认证；相较于单纯 Hash Chain，GMCP-R 还维护恢复票据和 checkpoint 相关状态。因此，GMCP-R 的吞吐量低于轻量基线，但其开销对应的是恢复后历史状态是否可信这一额外安全目标。

7.4 Concurrent Client Evaluation

并发实验测试 1、2、5、10 与 20 个客户端同时发送消息的情形。所有并发级别下，GMCP-R 均保持 100% 成功率，说明在当前测试规模内，协议的状态维护与服务器处理逻辑具有较好的稳定性。

表 9. 并发客户端测试结果

并发客户端数	成功率	总吞吐量 (msg/s)	平均 RTT (ms)	RTT 95% CI (ms)
1	100%	7,037	0.125	± 0.007

并发客户端数	成功率	总吞吐量 (msg/s)	平均 RTT (ms)	RTT 95% CI (ms)
2	100%	14,204	0.124	±0.004
5	100%	17,927	0.255	±0.007
10	100%	16,452	0.571	±0.015
20	100%	15,645	1.216	±0.018

数据来源：results/concurrent/concurrent_results.csv。

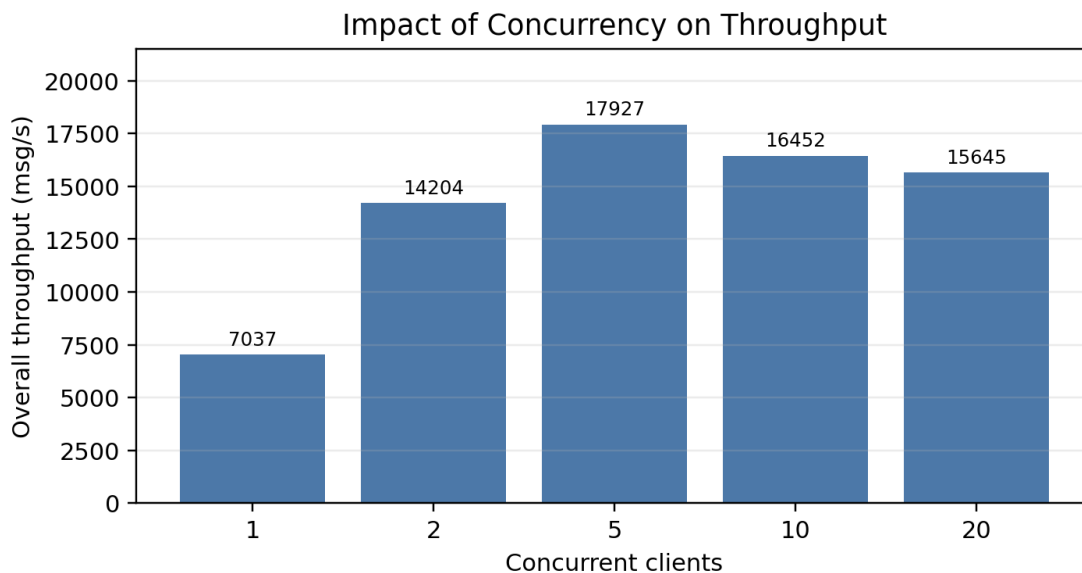


图 7. 并发客户端数量与总吞吐量变化。

数据来源：results/concurrent/concurrent_results.csv。

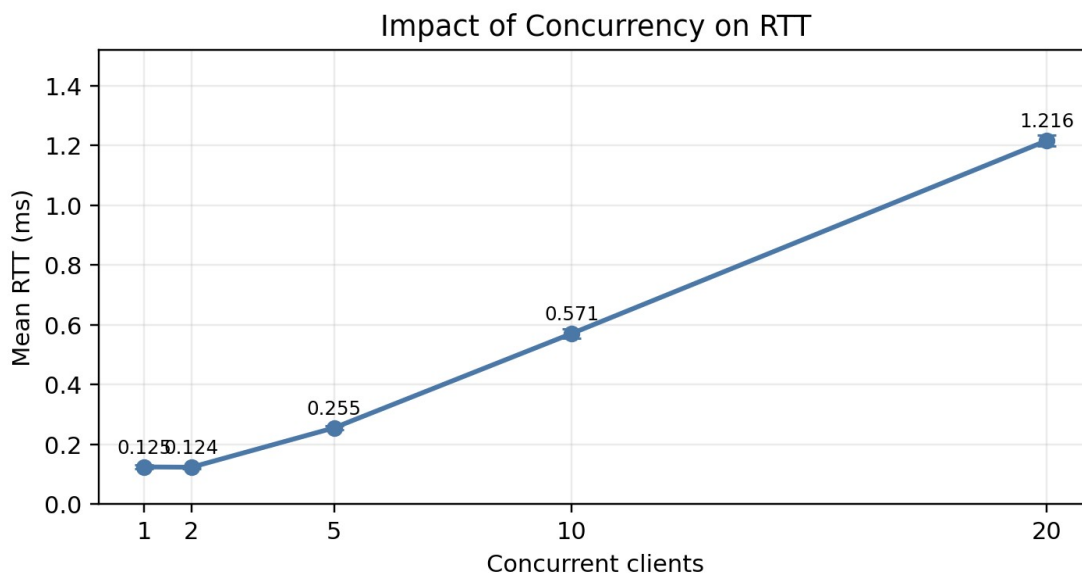


图 8. 并发客户端数量与平均 RTT 变化。

数据来源：results/concurrent/concurrent_results.csv。

并发结果显示，总吞吐量从 1 个客户端的 7,037 msg/s 增至 5 个客户端时的峰值 17,927 msg/s，随后在 10 和 20 个客户端下略有下降。该趋势与 Python 原型中的线程调度、GIL 影响、socket I/O 竞争以及服务端队列等待有关。平均 RTT 从 0.125 ms 上升到 1.216 ms，也反映了并发请求增加后的排队延迟和共享资源竞争。由于各并发级别成功率均为 100%，当前瓶颈主要表现为时延和吞吐变化，而非协议状态错误。

7.5 Preliminary Weak-Network Simulation

初步代码级弱网仿真用于观察当前实现中断连、丢包和恢复逻辑的行为。由于该实验尚未覆盖真实网络中的突发丢包、乱序、拥塞和带宽限制，本文不将其作为核心结论；相关表图仅作为补充实验列于 Appendix A，后续将使用 tc/netem 或 ns-3 进一步验证。

8. Discussion（讨论）

8.1 安全性与性能的权衡

GMCP-R 相比 Ticket Only 和 Seq+MAC 引入更多逐消息计算，但也获得了后两者不具备的历史连续性验证和安全恢复能力。对于高价值通信场景，6.6 万 msg/s 量级的本地处理能力通常不是主要瓶颈；实际系统中的网络时延、排队延迟和服务端并发处理能力更可能决定端到端性能。

8.2 与纯 Hash Chain 的关系

实验中 GMCP-R 与 Hash Chain 在攻击检测率上均表现为 100%，说明记忆链是历史完整性验证的关键。但 GMCP-R 的增量贡献在于恢复机制：MemoryTicket 将恢复点、last_mem 和 Checkpoint 状态以服务器签名方式绑定，Checkpoint 则降低长会话的恢复重放成本。换言之，GMCP-R 不是简单重复 Hash Chain，而是在其历史承诺能力上增加可验证、可恢复和可部署的恢复层。

8.3 适用场景

GMCP-R 适合对历史连续性要求高、但又可能出现断连或弱网恢复的系统，如工业设备遥测、远程控制、审计日志同步、金融交易流水同步、医疗记录传输和边缘 IoT 数据上报。对于只需短连接请求响应的轻量系统，GMCP-R 的额外开销可能并非必要。

8.4 Resource Constraints in IoT and Edge Devices

当前实验主要评估吞吐量、延迟、并发成功率和恢复行为，但 IoT 与边缘设备还需要关注能耗、SRAM 占用和持久化存储开销。GMCP-R 的逐消息计算开销主要来自 payload hash、memory state update 和 HMAC；在线状态主要包括 last_seq、last_mem、最近 checkpoint 以及已消费 nonce 记录。与保存完整消息历史相比，GMCP-R 只保留紧凑的链式状态和快照信息，因此状态存储开销更小，更接近轻量级应用层协议的部署需求。

不过，本文尚未在真实 MCU 或嵌入式平台上测试能耗、SRAM 峰值和持久化写入频率。后续工作将考虑在 ARM Cortex-M、Raspberry Pi 或工业网关平台上复现实验，以评估哈希、HMAC、checkpoint 写入和 nonce 管理在资源受限设备上的实际成本。

8.5 Limitations（局限性）

1. 当前实现主要使用对称密钥和 HMAC，开放环境中还需要与 TLS/QUIC 密钥协商或非对称签名机制集成。

- 2. 实验主要为单服务器架构，分布式多副本场景下的记忆状态一致性和票据跨节点验证仍需研究。
- 3. 弱网实验目前为代码级仿真，后续应使用 tc/netem 或 ns-3 对丢包、乱序、抖动和带宽限制进行更接近真实网络的评估。
- 4. 当前攻击场景覆盖 drop、modify、replay、prev_mem 和五类无效票据，仍需扩展到协商降级、选择性篡改、并发恢复竞争等复杂攻击。

8.6 Future Work: Post-Quantum Key Establishment Integration

GMCP-R 当前依赖 HMAC-SHA256 和 SHA-256 来实现消息认证、票据认证和记忆链更新；这些机制并不等同于完整的后量子安全通信方案。面向长期安全通信，未来可以将 GMCP-R 与 ML-KEM/Kyber 或混合密钥交换机制结合，用后量子或混合密钥建立过程产生会话密钥，再由 GMCP-R 在应用层维护历史连续性和恢复状态。该方向属于后续集成研究，本文不声称已经实现后量子安全。

9. Conclusions (结论)

本文提出 GMCP-R，一种面向断续通信的历史连续性安全恢复协议。该协议通过 memory chain 绑定消息历史，通过 MemoryTicket 验证恢复请求，通过 Checkpoint 控制恢复开销；恢复过程恢复的是 last_seq、last_mem 与 checkpoint 共同描述的历史记忆状态，而不是单纯恢复一条网络连接，从而缓解“连接恢复不等于历史状态恢复”的问题。

基于 Python TCP 原型的实验表明，在测试攻击场景下，GMCP-R 达到 100% 攻击检测率、0% 误接受率与 0% 误拒绝率；五类无效 MemoryTicket 均被拒绝；本地纯计算吞吐量约为 65,931 msg/s；1 至 20 个并发客户端均保持 100% 成功率。初步代码级弱网仿真仅作为补充证据，真实弱网结论仍需进一步验证。后续工作将集中在形式化验证、真实弱网评估、嵌入式资源测量、后量子密钥建立集成和分布式多副本恢复机制上。

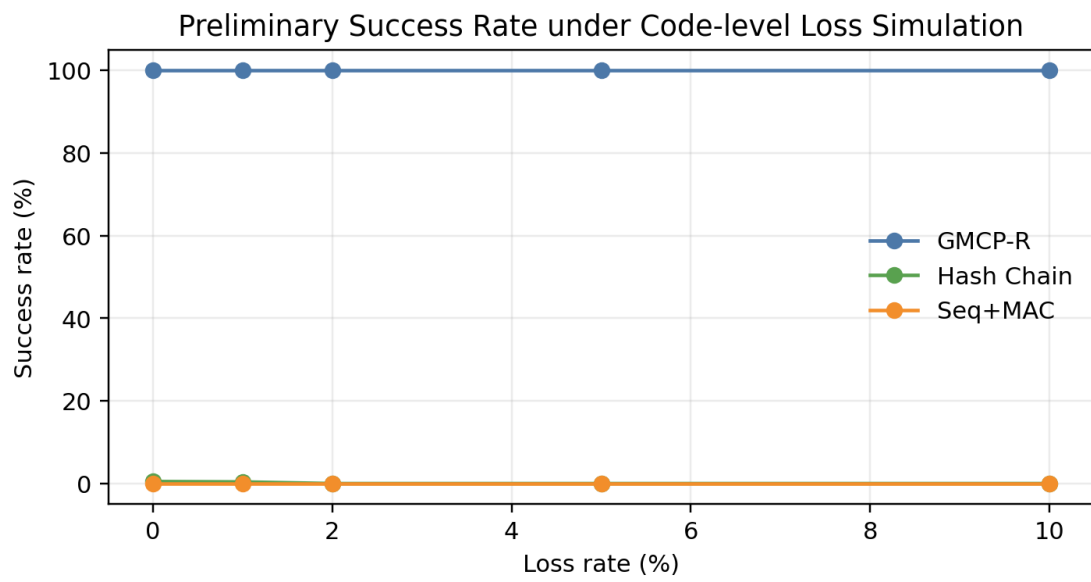
Appendix A. Weak-Network Supplement (补充弱网仿真)

本附录报告代码级弱网仿真的补充结果。该实验用于检查恢复机制在当前实现中的行为，不等同于操作系统级或网络仿真器级弱网评估，因此不作为本文核心结论。

附录表 A1. 弱网仿真初步结果 (50 次重复均值)

协议	成功率均值	成功率标准差	平均吞吐量 (msg/s)	平均 RTT (ms)
GMCP-R	100.0%	0.0%	72.81	79.90
Hash Chain	0.18%	0.24%	0.00	23.77
Seq+MAC	0.0%	0.0%	0.00	0.00

数据来源：results/weak_network_simulation/summary_weak_network_simulation.csv。



附录图 A1. 丢包率变化下的协议成功率。

数据来源：results/weak_network_simulation/weak_network_simulation_results.csv。